F. Glossary

#define — Chapter 2.1

A `#define` (pronounced "pound define") is a mechanism to expand macros in a program. This macro expansion occurs before the program is compiled. The following example expands the macro `PI` into 3.1415

```
#define PI 3.1415
```

#ifdef — Chapter 2.1

The `#ifdef` macro (pronounced "if-deaf") is a mechanism to conditionally include code in a program. If the condition is met (the referenced macro is defined), then the code is included.

```
#ifdef DEBUG
   cout << "I was here!\n";
#endif
```

abstract — Chapter 2.4

One of the three levels of understanding of an algorithm, abstract understanding is characterized by a grasp of how the parts or components of a program work together to produce a given output.

address-of operator — Chapter 3.3

The address-of operator (`&`) yields the address of a variable in memory. It is possible to use the address-of operator in front of any variable.

```
{
   int variable;
   cout << "The address of 'variable' is "
        << &variable;
}
```

ALU — Chapter 0.2

Arithmetic Logic Unit. This is the part of a CPU which performs simple mathematical operations (such as addition and division) and logical operations (such as `OR` and `NOT`)

argc — Chapter 4.3

When setting up a program to accept input parameters from the command line, argc is the traditional name for the number of items or parameters in the jagged array of passed data. The name "`argc`" refers to "count of arguments."

```
int main(int argc,          // count of parameters
         char ** argv);
```

argv — Chapter 4.3

When setting up a program to accept input parameters from the command line, `argv` is the traditional name for the jagged array containing the passed data. The name "`argv`" refers to "argument vector" or "list of unknown arguments."

```
int main(int argc,
         char ** argv);     // array of parameters
```

array	An array is a data-structure containing multiple instances of the same item. In other words, it is a "bucket of variables." Arrays have two properties: all instances in the collection are the same data-type and each instance can be referenced by index (not by name). `{ int array[4]; // a list of four integers array[2] = 42; // the 3rd member of the list }`	Chapter 3.0 Chapter 3.1
assembly	Assembly is a computer language similar to machine language. It is a low-level language lacking any abstraction. The purpose of Assembly language is to make Machine language more readable. Examples of Assembly language include `LOAD M:3` and `ADD 1`.	Chapter 0.2
`assert`	An assert is a function that tests to see if a particular assumption is met. If the assumption is met, then no action is taken. If the assumption is not met, then an error message is thrown and the program is terminated. Asserts are designed to only throw in debug code. To turn off asserts for shipping code, compile with the `-DNDEBUG` switch.	Chapter 2.1
bitwise operator	A bitwise operator is an operator that works on the individual bits of a value or a variable.	Chapter 3.5
`bool`	A `bool` is a built-in datatype used to describe logical data. The name "`bool`" came from the father of logical data, George Boole. `bool isMarried = true;`	Chapter 1.2 Chapter 1.5
Boolean operator	A Boolean operator is an operator that evaluates to a `bool` (`true` or `false`). For example, consider the expression (`value1 == value2`). Regardless of the data-type or value of `value1` and `value2`, the expression will always evaluate to `true` or `false`.	Chapter 1.5
data-driven	Data-driven design is a programming design pattern where most of the elements of the design are encoded in a data structure (typically an array) rather than in the algorithm. This allows a program to be modified without changing any of the code; only the data structure needs to be adjusted.	Chapter 3.1
`case`	A `case` label is part of a `switch` statement enumerating one of the options the program must select between.	Chapter 3.5
casting	The process of converting one data type (such as a `float`) into another (such as an `int`). For example, `(float)3` equals `3.0`.	Chapter 1.3
`char`	A `char` is a built-in datatype used to describe a character or glyph. The name "Char" came from "Character," being the most common use. `char letterGrade = 'B';`	Chapter 1.2

comments	Comments are notes placed in a program not read by the compiler.	Chapter 0.2
compiler	A compiler is a program to translate code in a one language (say C++) into another (say machine language).	Chapter 1.0

compound statement	A compound statement is a collection of statements grouped with curly braces. The most common need for this is inside the body of an IF statement or in a loop.
	`if (failed == true)`
	`{ // compound statement start`
	`  cout << "Sorry!\n"; // first statement`
	`  return false; // second statement`
	`} // compound statement end`
		Chapter 1.6

cohesion	The measure of the internal strength of a module. In other words, how much a function does one thing and one thing only. The four levels of cohesion are: Strong, Extraneous, Partial, and Weak.	Chapter 2.0
coincidental	A measure of cohesion where items are in a module simply because they happen to fall together. There is no relationship.	Chapter 2.0
communicational	A measure of cohesion where all elements work on the same piece of data.	Chapter 2.0
conceptual	One of the three levels of understanding of an algorithm, conceptual understanding is characterized by a high level grasp of what the program is trying to accomplish. This does not imply an understanding of what the individual components do or even how the components work together to produce the solution.	Chapter 2.4
concrete	One of the three levels of understanding of an algorithm, concrete understanding is characterized by knowing what every step of an algorithm is doing. It does not imply an understanding of how the various steps contribute to the larger problem being solved. The desk check tool is designed to facilitate a concrete understanding of code.	Chapter 2.4
conditional expression	A conditional expression is a decision mechanism built into C++ allowing the programmer to choose between two expression, rather than two statements. `cout << (grade >= 60.0 ? "pass" : "fail");`	Chapter 3.5
control	A measure of coupling where one module passes data to another that is interpreted as a command.	Chapter 2.0

counter-controlled	One of the three loop types, a counter-controlled loop keeps iterating a fixed number of types. Typically, this number is known when the loop begins. A counter-controlled loop has four components: the start, the end, the counter, and a loop body.	Chapter 2.5
coupling	Coupling is the measure of information interchange between functions. The seven levels of coupling are: Trivial, Encapsulated, Simple, Complex, Document, Interactive, and Superfluous.	Chapter 2.0
`cout`	COUT stands for Console OUTput. Technically speaking, `cout` a the destination or output stream. In other words, it in the following example, it is the destination where the insertion operator (`<<`) is sending data to. In this case, that destination is the screen. `cout << "Hello world!";`	Chapter 1.1
CPU	Central Processing Unit. This is the part of a computer that interprets machine-language instructions	Chapter 0.2
c-string	A c-string is how strings are stored in C++: an array of characters terminated with a null (`'\0'`) character.	Chapter 3.2
data	A measure of coupling where the data passed between functions is very simple. This occurs when a single atomic data item is passed, or when highly cohesive data items are passed	Chapter 2.0
decoder	The instruction decoder is the part of the CPU which identifies the components of an instruction from a single machine language instruction.	Chapter 0.2
`default`	A `default` label is a special `case` label in a `switch` statement corresponding to the "unknown" or "not specified" condition. If none of the `case` labels match the value of the controlling expression, then the `default` label is chosen.	Chapter 3.5

`delete`	The `delete` operator serves to free memory previously allocated with `new`. When a variable is declared on the stack such as a local variable, this is unnecessary; the operating system deletes the memory for the user. However, when data is allocated with new, it is the programmer's responsibility to delete his memory.
	`{`
	`  int * pValue = new int;`
	`  delete pValue;`
	`}`
		Chapter 4.1

DeMorgan	Just as there are ways to manipulate algebraic equations using the associative and distributed properties, it is also possible to manipulate Boolean equations. One of these transformations is DeMorgan. A few DeMorgan equivalence relationships are:
	`!(p || q) == !p && !q`
	`!(p && q) == !p || !q`
	`a || (b && c) == (a || b) && (a || c)`
	`a && (b || c) == (a && b) || (a && c)`
		Chapter 1.5

dereference operator	The dereference operator '*' will retrieve the data refered to by a pointer. `cout << "The data in the variable pValue is " << *pValue;`	Chapter 3.3
desk check	A desk check is a technique used to predict the output of a given program. It accomplished by creating a table representing the value of the variables in the program. The columns represent the variables and the rows represent the value of the variables at various points in the program execution.	Chapter 2.4
`do … while`	One of the three types of loops, a DO-WHILE loop continues to execute as long as the condition in the Boolean expression evaluates to true. This is the same as a WHILE loop except the body of the loop is guaranteed to be executed at least once. `do cin >> gpa; while (gpa > 4.0 || gpa < 0.0);`	Chapter 2.3
`double`	A `double` is a built-in datatype use to describe large read numbers. The word "Double" comes from "Double-precision floating point number," indicating it is just like a `float` except it can represent a larger number more precisely. `double pi = 3.14159265359;`	Chapter 1.2
driver	A driver is a program designed to test a given function. Usually a driver has a collection of simple prompts to feed input to the function being tested, and a simple output to display the return values of the function.	Chapter 2.1
dynamically-allocated array	A dynamically-allocated array is an array that is created at run-time rather than at compile time. Stack arrays have a size known at compile time. Dynamically-allocated arrays, otherwise known as heap arrays, can be specified at run-time. `{ int * array = new int[size]; }`	Chapter 4.1
`endl`	ENDL is short for "END of Line." It is one of the two ways to specify that the output stream (such as `cout`) will put a new line on the screen. The following example will put two blank lines on the screen: `cout << endl << endl;`	Chapter 1.1
`eof`	When reading data from a file, one can detect if the end of the file is reached with the `eof()` function. Note that this will only return `true` if the end of file marker was reached in the last read. `if (fin.eof()) cout << "The end of the file was reached\n";`	Chapter 2.6

escape sequences	Escape sequences are simply indications to `cout` that the next character in the output stream is special. Some of the most common escape sequences are the newline (`\n`), the tab (`\t`), and the double-quote (`\"`)	Chapter 1.1
event-controlled	One of the three loop types, an event-controlled loop is a loop that keeps iterating until a given condition is met. This condition is called the event. There are two components to an event-controlled loop: the termination condition and the body of the loop.	Chapter 2.5
expression	A collections of values and operations that, when evaluated, result in a single value. For example, `3 * value` is an expression. If value is defined as `float value = 1.5;`, then the expression evaluates to `4.5`.	Chapter 1.3
external	A measure of coupling where two modules communicate through a global variable or another external communication avenue.	Chapter 2.0
extraction operator	The extraction operator (`>>`) is the operator that goes between `cin` and the variable receiving the user input. In the following example, the extraction operator is after the `cin`. `cin >> data;`	Chapter 1.2
fetcher	The instruction fetcher is the part of the CPU which remembers which machine instruction is to be retrieved next. When the CPU is ready for another instruction, the fetcher issues a request to the memory interface for the next instruction.	Chapter 0.2

`for`	One of the three types of loops, the FOR loop is designed for counting. It contains fields for the three components of most counting problems: where to start (the Initialization section), where the end (the Boolean expression), and what to change with every count (the Increment section).
	`for (int i = 0; i < num; i++)`
	`  cout << array[i] << endl;`
		Chapter 2.3

`fstream`	The `fstream` library contains tools enabling the programmer to read and write data to a file. The most important components of the `fstream` library are the `ifstream` and `ofstream` classes. `#include <fstream>`	Chapter 2.6
function	One division of a program. Other names are sub-routine, sub-program, procedure, module, and method.	Chapter 1.4
functional	A measure of cohesion where every item in the function is related to a single task.	Chapter 2.0

`getline`	The `getline()` method works with `cin` to get a whole line of user input.
	`char text[256];  // getline needs a string`
	`cin.getline(text, 256);  // the size is a parameter`
		Chapter 1.2

global variable	A global variable is a variable defined outside a function. The scope extends to the bottom of the file, including any function that may be defined below the global. It is universally agreed that global variables are evil and should be avoided.	Chapter 1.4

`ifstream`	The `ifstream` class is part of the `fstream` library, enabling the programmer to write data to a file. IFSTREAM is short for "Input File STREAM."
	`#include <fstream>`
	`{`
	`  ifstream fin("file.txt");`
	`  …`
	`}`
		Chapter 2.6

insertion operator	The insertion operator (`<<`) is the operator that goes between `cout` and the data to be displayed. As we will learn in CS 165, the insertion operator is actually the function and `cout` is the destination of data. In the following example, the insertion operator is after the `cout`. `cout << "Hello world!";`	Chapter 1.1
instrumentation	The process of adding counters or markers to code to determine the performance characteristics. The most common ways to instrument code is to track execution time (by noting start and completion time of a function), iterations (by noting how many times a given block of code has executed), and memory usage (by noting how much memory was allocated during execution).	Chapter 4.4
`int`	An `int` is a built-in datatype used to describe integral data. The word "Int" comes from "Integer" meaning "all whole numbers and their opposites." `int age = 19;`	Chapter 1.2
`iomanip`	The IOMANIP library contains the `setw()` method, enabling a C++ program to right-align numbers. The programmer can request the IOMANIP library by putting the following code in the program: `#include <iomanip>`	Chapter 1.1
`iostream`	The IOSTREAM library contains `cin` and `cout`, enabling a simple C++ program to display text on the screen and gather input from the keyboard. The programmer can request IOSTREAM by putting the following code in the program: `#include <iostream>`	Chapter 0.2

jagged array	A jagged array is a special type of multi-dimensional array where each row could be of a different size.	Chapter 4.3
lexer	The lexer is the part of the compiler to break a program into a list of tokens which will then be parsed.	Chapter 1.0
local variable	A local variable is a variable defined in a function. The scope of the variable is limited to the bounds of the function.	Chapter 1.4
logical	A level of cohesion where items are grouped in a module because they do the same kinds of things. What they operate on, however, is totally different.	Chapter 2.0
machine	Machine language is a computer language understandable by a CPU. It is language of the lowest abstraction. Machine language consists of noting but 1's and 0's.	Chapter 0.2
modularization	Modularization is the process of dividing a problem into separate tasks, each with a single purpose.	Chapter 2.0
modulus	The remainder from division. Consider 14 ÷ 3. The answer is 4 with a remainder of 2. Thus fourteen modulus 3 equals 2: 14 % 3 == 2	Chapter 1.3

multi-dimensional array	A multi-dimensional array is an array of arrays. Instead of accessing each member with a single index, more than one index is required. The following is a multi-dimensional array representing a tic-tac-toe board:
	`{`
	`  char board[3][3];`
	`}`
		Chapter 4.0

multi-way IF	Though an IF statement only allows the programmer to distinguish between at most two options, it is possible to specify more options through the use of more than one IF. This is called an multi-way IF.
	`if (grade >= 90.0)`
	`  cout << "A";  // first condition`
	`else if (grade >= 90.0)`
	`  cout << "B";  // second condition`
	`else`
	`  cout << "not so good!";  // third condition`
		Chapter 1.6

nested statement	A nested statement is a statement inside the body of another statement. For example, an IF statement inside the body of another IF statement would be considered a nested IF.
	`if (grade >= 80.0)  // outer IF statement`
	`  if (grade >= 90)  // nested IF statement`
	`  cout << 'A';  // body of nested IF statement`
	`  else`
	`  cout << 'B';`
		Chapter 1.6

`new`	It is possible to allocate a block of memory with the `new` operator. This serves to issue a request to the operating system for more memory. It works with single items as well as arrays. `{ int * pValue = new int; // one integer int * array = new int[10]; // ten integers }`	Chapter 4.1
null	The null character, also known as a null terminator, is a special character marking the end of a c-string. The null character is represented as `'\0'`, which is always defined as zero. `{ char nullCharacter = '\0'; // 0x00 }`	Chapter 3.2
`NULL`	The `NULL` address corresponds to the zero address `0x00000000`. This address is guaranteed to be invalid, making it a convenient address to assign to a pointer when the pointer variable does not point to anything. `{ int * pValue = NULL; // points to nothing }`	Chapter 4.1
`ofstream`	The `ofstream` class is part of the `fstream` library, enabling the programmer to write data to a file. OFSTREAM is short for "Output File STREAM." `#include <fstream> { ofstream fout("file.txt"); … }`	Chapter 2.6
online desk check	An online desk check is a technique to gain an understanding of how data flows through an existing program. This is accomplished by putting COUT statements at strategic places in a program to display the value of key variables.	Chapter 2.4
parser	The parser is the part of the compiler understanding the syntax or grammar of the language. Knowing this, it is able to take all the components from the input language and place it into the format of the target or output language.	Chapter 1.0
Pascal-string	One of the two main implementations of strings, a Pascal-string is an array of characters where the length is stored in the first slot. This is not how strings are implemented in C++.	Chapter 3.2
pass-by-reference	Pass-by-reference, also known as "call-by-reference," is the process of sending a parameter to a function where the caller and the callee share the same variable. This means that changes made to the parameter in the callee will be reflected in the caller. You specify a pass-by-reference parameter with the ampersand `&`. `void passByReference(int &parameter);`	Chapter 1.4 Chapter 3.3

pass-by-pointer	Pass-by-pointer, more accurately called "passing a pointer by value," is the process of passing an address as a parameter to a function. This has much the same effect as pass-by-reference. `void passByPointer(int * pParameter);`	Chapter 3.3
pass-by-value	Pass-by-value, also known as "call-by-value," is the process of sending a parameter to a function where the caller and the callee have different versions of a variable. Data is sent one-way from the caller to the callee; no data is sent back to the caller through this mechanism. This is the default parameter passing convention in C++. `void passByValue(int parameter);`	Chapter 1.4 Chapter 3.3
pointer	A pointer is a variable holding and address rather than data. A data variable, for example, may hold the value 3.14159. A pointer variable, on the other hand, will contain the address of some place in memory.	Chapter 3.3
procedural	A measure of cohesion where all related items must be performed in a certain order.	Chapter 2.0
prototype	A prototype is the name, parameter list, and return value of a function to be defined later in a file. The purpose of the prototype is to give the compiler "heads-up" as to which functions will be defined later in the file.	Chapter 1.4

pseudocode	Pseudocode is a high-level programming language designed to help people design programs. Though it has most of the elements of a language like C++, pseudocode cannot be compiled. An example of pseudocode is:
	`computeTithe(income)`
	`  RETURN income ÷ 10`
	`END`
		Chapter 2.2

register	The part of a CPU which stores short-term data for quick recall. A CPU typically has many registers.	Chapter 0.2
sentinel-controlled	One of the three loop types, a sentinel-controlled loop keeps iterating until a condition is met. This condition is controlled by a sentinel, a Boolean variable set by a potentially large number of divergent conditions.	Chapter 2.5
sequential	A measure of cohesion where operations in a module must occur in a certain order. Here operations depend on results generated from preceding operations	Chapter 2.0
scope	Scope is the context in which a given variable is available for use. This extends from the point where the variable is defined to the next closing braces }.	Chapter 1.4

`sizeof`	The `sizeof` function returns the number of bytes that a given datatype or variable requires in memory. This function is unique because it is evaluated at compile-time where all other functions are evaluated at run-time.
	`{`
	`  int integerVariable;`
	`  cout << sizeof(integerVariable) << endl;  // 4`
	`  cout << sizeof(int)  << endl;  // 4`
	`}`
		Chapter 1.2
		Chapter 3.0

stack variable	A stack variable, otherwise known as a local variable, is a variable that is created by the compiler when it falls into scope and destroyed when it falls out of scope. The compiler manages the creation and destruction of stack variables wherease the programmer manages the createion and destruction of dynamically allocated (heap) variables.	Chapter 4.1
stamp	A measure of coupling where complex data or a collection of unrelated data items are passed between modules.	Chapter 2.0

string	A "string" is a computer representation of text. The term "string" is short for "an alpha-numeric string of characters." This implies one of the most important characteristics of a string: is a sequence of characters. In C++, a string is defined as an array of characters terminated with a null character.
	`{`
	`  char text[256];  // a string of 255 characters`
	`}`
		Chapter 1.2
		Chapter 3.2

structure chart	A structure chart is a design tool representing the way functions call each other. It consists of three components: the name of the functions of a program, a line connecting functions indicating one function calls another, and the parameters that are passed between functions.	Chapter 2.0
`styleChecker`	`styleChecker` is a program that performs a first-pass check on a student's program to see if it conforms to the University style guide. The `styleChecker` should be run before every assignment submission.	Chapter 1.0 Appendix A

stub	A stub function is a placeholder for a function that is not written yet. The closest analogy is an outline in an essay: a placeholder for a chapter or paragraph to be written later. An example stub function is:
	`void display(float value)`
	`{`
	`}`
		Chapter 2.1

`submit`	`submit` is a program to send a student's file to the appropriate instructor. It works by reading the program header and, based on what is found, sending it to the instructor's class and assignment directory.	Chapter 1.0
`switch`	A `switch` statement is a mechanism built into most programming langauges allowing the programmer to specify between more than two options.	Chapter 3.5

tabs	The tab key on a traditional typewriter was invented to facilitate creating tabular data (hence the name). The tab character (`'\t'`) serves to move the cursor to the next tab stop. By default, that is the next 8 character increment. `cout << "\tTab";`	Chapter 1.1
temporal	A measure of cohesion where items are grouped in a module because the items need to occur at nearly the same time. What they do or how they do it is not important	Chapter 2.0
`testBed`	`testBed` is a tool to compare a student's solution with the instructor's key. It works by compiling the student's assignment and running the program against a pre-specified set of input and output.	Chapter 1.0
variable	A variable is a named location where you store data. The name must be a legal C++ identifier (comprising of digits, letters, and the underscore _ but not starting with a digit) and conform to the University style guide (camelCase, descriptive, and usually a noun). The location is determined by the compiler, residing somewhere in memory.	Chapter 1.2

`while`	One of the three types of loops, a WHILE-loop continues to execute as long as the condition inside the Boolean expression is true.
	`while (grade < 70)`
	`  grade = takeClassAgain();`
		Chapter 2.3